Laboratorio Calificado 13

2026-1

ÍNDICE

1 Instrucciones 3
2 Contexto 3
3 Objetivos 4
4 Requisitos 4
5 Entregables 7
6 Rúbrica de Evaluación 7

Laboratorio Calificado 13

Registro de inspecciones de calidad con transacciones, modelado de clases, GUI y base de datos para el Sistema de Gestión de Ensamblaje de Unidades Robóticas

1 Instrucciones

Se ha creado la tarea “Laboratorio Calificado 13” en la sección correspondiente de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con los entregables detallados más adelante. El nombre del archivo comprimido deberá tener el siguiente formato:

`L13_<Código del alumno de 8 dígitos>.zip`

Como ejemplo, el nombre de archivo del alumno 20236969 sería “L13_20236969.zip”. Es completa responsabilidad de cada alumno respetar el formato de entrega para evitar problemas en la calificación.

2 Contexto

En el Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica, cada lote de producción procesado por las **Unidades Robóticas** debe someterse a una rigurosa inspección de calidad antes de avanzar al siguiente proceso de la cadena de manufactura. Esto garantiza que las calibraciones cinemáticas y los movimientos del brazo mecánico se mantengan dentro de los márgenes operativos nominales.

Cada registro de control de calidad está estructurado bajo un esquema **Maestro-Detalle**, compuesto por:

- **Cabecera de inspección (InspeccionCabecera):** Almacena los metadatos generales de la evaluación (ID de la inspección, código del lote afectado, ID de la **Unidad Robótica** encargada del procesamiento, operario inspector responsable, fecha de la prueba y el resultado general del lote: *"Aprobado"* o *"Observado"*).
- **Uno o varios detalles (InspeccionDetalle):** Desglosa las métricas físicas específicas evaluadas sobre los componentes mecánicos producidos (e.g., *"Tolerancia Dimensional (mm)"*, *"Acabado Superficial"*, *"Alineación de Actuadores"*), guardando el veredicto del parámetro (*"OK"* / *"NO OK"*) junto con observaciones opcionales.

Para asegurar la integridad de los datos industriales en el servidor de la planta, el registro debe ejecutarse como una **transacción atómica completa**, bajo las siguientes restricciones de robustez:

- Si la cabecera se inserta correctamente pero el registro de un solo parámetro de detalle llega a fallar debido a un error de red o de integridad, el sistema debe efectuar un **ROLLBACK** revirtiendo todo el proceso para evitar dejar datos huérfanos.
- Si la totalidad de los componentes y tareas desglosadas se procesan con éxito, se ejecuta un **COMMIT** que confirma la transacción.

Toda la persistencia de datos y el flujo operativo de calidad serán administrados de forma centralizada en **SQL Server** a través de procedimientos almacenados parametrizados.

3 Objetivos

- **Modelar e implementar las clases orientadas a objetos** InspeccionCabecera e InspeccionDetalle dentro de la capa de Modelo (Model), representando la jerarquía estructural de las auditorías de manufactura.
- **Ejecutar y analizar un script SQL transaccional** en el servidor para instanciar las tablas y los procedimientos almacenados orientados al control de calidad de la planta.
- **Consolidar la conectividad relacional** de la aplicación en C++/CLI con SQL Server, aislando la lógica de acceso a datos dentro de la capa de Persistencia (Persistence).
- **Diseñar un formulario interactivo Maestro-Detalle** en la capa de Presentación (View) utilizando Windows Forms para capturar las evaluaciones físicas de los lotes de producción.
- **Garantizar la integridad referencial y de datos** mediante el registro atómico de la información en SQL Server utilizando transacciones y procedimientos almacenados parametrizados.

4 Requisitos

a) Conexión a la base de datos

El estudiante deberá conectarse a:

- Su propia instancia de SQL Server (AWS), o
- La base de datos del curso (credenciales enviadas por correo electrónico).

b) Script inicial (provisto por el docente)

El siguiente script debe ejecutarse en SQL Server:

Sql

```
-- Verificar y eliminar tablas existentes respetando el orden de dependencia (primero el detalle)
IF OBJECT_ID('dbo.InspeccionDetalle', 'U') IS NOT NULL
    DROP TABLE InspeccionDetalle;
GO

IF OBJECT_ID('dbo.InspeccionCabecera', 'U') IS NOT NULL
    DROP TABLE InspeccionCabecera;
GO

-- ============================================================
-- 1. CREACIÓN DE TABLAS DE CONTROL DE CALIDAD (MAESTRO - DETALLE)
-- ============================================================

-- Tabla Maestro: Cabecera de Inspección de Calidad del Lote
CREATE TABLE InspeccionCabecera (
    InspeccionId INT PRIMARY KEY IDENTITY(1,1),
    CodigoLote NVARCHAR(50),
    RobotId INT,                                    -- Enlace al brazo robótico responsable de
la manufactura
    Inspector NVARCHAR(100),
    FechaInspeccion DATETIME DEFAULT GETDATE(),
    ResultadoGeneral NVARCHAR(20),                  -- 'Aprobado' o 'Observado'
    FOREIGN KEY (RobotId) REFERENCES UnidadRobotica(RobotId)
);
GO

-- Tabla Detalle: Parámetros específicos de tolerancia evaluados
CREATE TABLE InspeccionDetalle (
    InspeccionDetalleId INT PRIMARY KEY IDENTITY(1,1),
    InspeccionId INT,
    Parametro NVARCHAR(100),                        -- e.g., 'Tolerancia Dimensional', 'Acabado
Superficial'
    ResultadoParametro NVARCHAR(10),                -- 'OK' o 'NO OK'
    Notas NVARCHAR(200),                            -- Observaciones técnicas opcionales
    FOREIGN KEY (InspeccionId) REFERENCES InspeccionCabecera(InspeccionId)
);
GO
```

```
-- ============================================================
-- 2. PROCEDIMIENTOS ALMACENADOS (STORED PROCEDURES)
-- ============================================================

-- Procedimiento para registrar la Cabecera (Retorna el ID autogenerado)
IF OBJECT_ID('dbo.usp_CreateInspeccion', 'P') IS NOT NULL
    DROP PROCEDURE usp_CreateInspeccion;
GO

CREATE PROCEDURE usp_CreateInspeccion
    @CodigoLote NVARCHAR(50),
    @RobotId INT,
    @Inspector NVARCHAR(100),
    @ResultadoGeneral NVARCHAR(20),
    @InspeccionId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO InspeccionCabecera (CodigoLote, RobotId, Inspector, ResultadoGeneral)
    VALUES (@CodigoLote, @RobotId, @Inspector, @ResultadoGeneral);

    -- Captura el ID incremental generado en esta transacción
    SET @InspeccionId = SCOPE_IDENTITY();
END;
GO

-- Procedimiento para insertar cada Parámetro de Control en el Detalle
IF OBJECT_ID('dbo.usp_AddInspeccionDetail', 'P') IS NOT NULL
    DROP PROCEDURE usp_AddInspeccionDetail;
GO

CREATE PROCEDURE usp_AddInspeccionDetail
    @InspeccionId INT,
    @Parametro NVARCHAR(100),
    @ResultadoParametro NVARCHAR(10),
    @Notas NVARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO InspeccionDetalle (InspeccionId, Parametro, ResultadoParametro, Notas)
    VALUES (@InspeccionId, @Parametro, @ResultadoParametro, @Notas);
END;
GO
```

c) Actividades obligatorias del estudiante

El estudiante debe implementar la solución estructurada en las siguientes capas arquitectónicas:

1. En la capa del Modelo (Model) — OBLIGATORIO

Modelado orientado a objetos de las clases que representan la jerarquía del control de calidad:

- **Clase InspeccionCabecera (Cabecera):**
    - **Atributos mínimos requeridos:**
        - int inspeccionId;
        - String^ codigoLote;
        - int robotId; *(Identificador de la unidad robótica responsable de la manufactura)*
        - String^ inspector;
        - String^ resultadoGeneral; *(e.g., Aprobado, Observado)*
        - DateTime fechaInspeccion;

        - List<InspeccionDetail^>^ detalles; *(Lista dinámica de parámetros evaluados)*

- **Clase InspeccionDetalle (Detalle):**
    - **Atributos mínimos requeridos:**
        - int inspeccionDetailId;
        - String^ parametro; *(e.g., Tolerancia Dimensional, Acabado)*
        - String^ resultadoParametro; *(e.g., OK, NO OK)*
        - String^ notas;
    - **Requisitos:** Implementar los constructores apropiados para inicializar los datos de auditoría de cada componente.

2. En la capa de Persistencia (Persistence)

Implementar los métodos de conectividad relacional con ADO.NET encargados de comunicarse con SQL Server para:

- **Ejecutar usp_CreateInspeccion:** Insertar el registro principal y recuperar el InspeccionId autogenerado por el servidor como parámetro de salida.
- **Ejecutar usp_AddInspeccionDetail:** Registrar iterativamente cada parámetro evaluado en la grilla.
- **Manejo de robustez industrial:** Gestionar de manera adecuada las excepciones SQL (SqlException), estructurar el control de transacciones en la base de datos y asegurar la notificación limpia de errores hacia la capa superior.

3. En la capa de Control o Servicio (Controller)

Desarrollar la lógica de negocio que orqueste el proceso:

- Recibir una instancia completa de InspeccionCabecera con su lista interna de InspeccionDetalle debidamente poblada desde la interfaz gráfica.
- **Validar los datos de planta:** Asegurar la integridad de las cadenas y que las reglas de negocio se cumplan (por ejemplo, consistencia en los IDs y códigos).
- **Ejecutar la transacción atómica:** Invocar los métodos de persistencia garantizando el ciclo completo de almacenamiento relacional.

4. En la capa de Presentación (View – C++/CLI)

Desarrollar e integrar el formulario interactivo en Windows Forms para el operador de control de calidad, el cual debe permitir:

- Ingresar el código del lote de producción bajo análisis.
- Seleccionar o ingresar la **Unidad Robótica** encargada y el inspector responsable de la auditoría.
- Definir el resultado global del lote (*Aprobado* / *Observado*).
- **Grilla Dinámica de Parámetros:** Permitir al usuario agregar múltiples filas con las evaluaciones específicas dentro de un **DataGridView**, especificando:
    - Nombre del parámetro técnico evaluado.
    - Resultado del parámetro (*"OK"*, *"NO OK"*).
    - Notas u observaciones de tolerancia (opcional).
- **Procesamiento:** Registrar formalmente la inspección integral mediante la pulsación de un botón dedicado.

- **Retroalimentación:** Desplegar cuadros de diálogo claros indicando la confirmación del almacenamiento exitoso en el servidor o el mensaje detallado del error capturado.

5 Entregables

El archivo ZIP deberá contener:

- Código fuente completo en C++/CLI, estructurado en capas (Modelo, Persistencia, Servicio o Controlador, Vista), con comentarios apropiados.
- Capturas de pantalla que evidencien:
  - El formulario funcionando: ingreso de mantenimiento, ingreso de datos y ejecución.
  - La ejecución exitosa del registro del mantenimiento.

6 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente (100%)	Bueno (75%)	Regular (50%)	Deficiente (25%)	No Cumple (0%)
Modelado de Inspection y InspectionDetail (4 pts)	Clases completas y bien diseñadas.	Omisiones menores.	Diseño incompleto.	Diseño deficiente.	No implementado.
Persistencia – SP + transacción (6 pts)	Lógica robusta y validada.	Pequeños errores.	Parcial o poco validado.	Múltiples errores.	No implementado.
Controlador – integración (4 pts)	Flujo correcto, modular.	Fallos menores.	Parcial.	Flujo pobre.	Nulo.
Formulario – Presentación (4 pts)	Interfaz clara, validada y funcional.	Detalles menores.	Incompleta.	Poco usable.	No implementada.
Documentación y capturas (2 pts)	Clara, completa.	Leves omisiones.	Incompleta.	Deficiente.	No presentada.